

Modelagem de Software

Paradigma Imperativo

- A ideia geral é o uso de subprogramação
 - Agrupamento de atividades simples para gerar atividades complexas
 - Nomeação desses blocos de atividades
 - Chamar o bloco de atividades pelo nome em qualquer ponto
- Essas ações complexas são chamadas de procedimentos, subrotinas ou funções

Paradigma Orientado a Objetos (OO)

.Classes de Objetos

- Agrupamento de funções e variáveis afins**

.Pacotes de Classes

- Agrupamento de classes afins**

- Bibliotecas de apoio**

Imperativo x Orientado Objetos

- O paradigma imperativo organiza em termos de algoritmos
- O paradigma orientado a objetos organiza em termos de objetos

Objetos

- Podemos criar programas pensando em objetos ao invés de algoritmos?
- O mundo é composto de objetos
 - Uma loja tem produtos, pedidos, estoque, etc.
 - Uma escola tem professores, alunos, disciplinas, etc.
- E se criarmos programas basicamente criando objetos equivalentes ao do mundo real, fazendo com que eles se comuniquem?

Objetos

- Um objeto é a representação computacional de um elemento ou processo do mundo real
- Cada objeto possui suas características e comportamentos

Características dos Objetos

- .Uma característica descreve uma propriedade do objeto, ou seja, um elemento que o compõe.**
- .Cada característica é chamada de atributo e funciona como uma variável que pertence ao objeto.**

Comportamentos de Objetos

- Um comportamento é uma ação ou resposta de um objeto a uma ação do mundo real.
- Cada comportamento é chamado de método e funciona como um procedimento, ou função, pertencente ao objeto.

Traduzindo

.Mundo Real

- Características
- Comportamentos

.Mundo Computacional

- Atributos
- Métodos

Exemplo: Agenda

.Paradigma Imperativo

- Variáveis

- .Vetor de nomes

- .Vetor de endereços

- .Vetor de telefones

- Procedimentos

- .Listar nomes

- .Exibir endereço dado nome

- Exibir telefone dado nome

.Orientado a Objetos

- Objeto Agenda

- .Atributos

- Vetor de contatos

- .Métodos

- Listar contatos

- Buscar contato dado Nome

- Adicionar contato

- Remover contato

Exemplo: Compra

- .O objeto “Compra” é formado por “Itens”.
- .Os objetos “Itens” são formados por “produtos” e “quantidades”
- .Os objetos “produto” possuem “valor unitário”

Princípios do Paradigma OO

- **Abstração**
- **Encapsulamento**
- **Modularidade**
- **Hierarquia**

Abstração

.O objeto computacional deve se concentrar nas propriedades e métodos que são relevantes para o problema em questão.

.O objeto carro, por exemplo, quais seriam os atributos e métodos nas seguintes aplicações:

- Locadora de veículos**
- Revendedora de carros**
- Oficina Mecânica**
- DETRAN**

Encapsulamento

- .O objeto deve esconder seus dados e detalhes de implementação.
- .Os métodos formam uma “cerca” ao redor dos atributos.
 - Atributos não devem ser manipulados diretamente
 - Atributos devem ser consultados e alterados através de métodos

Encapsulamento

.Alguns métodos são públicos, podendo ser utilizados por outros objetos, e são a forma de comunicação entre os objetos.

.Alguns métodos são privados, que cuidam do funcionamento e gerenciamento interno do objeto.

Modularidade

- Consiste em dividir o todo em partes bem definidas, que podem ser construídas e examinadas separadamente.
- Essas partes interagem entre si, fazendo o sistema funcionar de forma adequada.
- Um programa OO é um conjunto de objetos que colaboram entre si para resolver o problema.
- A colaboração se dá através das chamadas de métodos.
 - Ex: O Motorista freia o Carro

Hierarquia

- Objetos herdam atributos e métodos dos seus ancestrais.
- A herança é um mecanismo para simplificar a definição de Classes de objetos similares
 - Qual a semelhança entre um leão, um tigre e um gato?
 - Qual a semelhança destes com um lobo e um cachorro?
 - Qual a semelhança de todos estes com uma baleia?

Classe x Objeto

- A Classe é o tipo do objeto.
 - Valores têm tipos primitivos (inteiro, real, lógico)
 - Objetos pertencem a classes
- Uma classe é uma forma, capaz de produzir objetos.
- Nós criamos as classes, as classes instanciam os objetos.
- Elas descrevem as características e comportamentos de um conjunto de objetos.

Exemplo de Classe

.Vamos relembrar o que já vimos na primeira disciplina e tentar criar classes para os exemplos que foram citados até agora.

Herança entre Classes

- Para viabilizar a hierarquia de objetos, as classes possuem uma estrutura hierárquica.
- A classe que fornece elementos, a antecessora, é chamada de superclasse.
- A classe herdeira é chamada de subclasse
- A subclasse pode herdar métodos e atributos de mais de uma superclasse
- A subclasse pode ter métodos e atributos novos

Exemplo

- .Um “Veículo” pode ser uma Bicicleta, um Automóvel, um Navio, ou um Avião.
- .Um “Automóvel” pode ser um Carro, Caminhão, Trator, etc.

Exercício

.Identifique as classes

–O supermercado vende diferentes tipos de produtos. Cada produto tem um preço e uma quantidade em estoque. Um pedido de um cliente é composto de itens, onde cada item especifica o produto que o cliente deseja e a respectiva quantidade. Esse pedido pode ser pago em dinheiro, pix ou cartão

*Interfaces

- Uma interface é um tipo especial de classe que não tem implementação
 - Define um protocolo
 - Classes podem implementar uma ou mais interfaces
- Uma interface é um contrato assinado por uma classe
 - A interface define as responsabilidades de quem a implementa
 - As responsabilidades são métodos
 - A classe que implementa uma interface implementa seus métodos
 - A interface contém somente a assinatura de métodos e constantes

*Interfaces

.Exemplo:

–A interface taxavel tem:

.Atributo: Ano_Inicio = 1996

.Método: calculaTaxa();

*Classes Abstratas

- .Classes que não podem ser instanciadas
- .Esboçam a estrutura de subclasses
- .Podem ter métodos abstratos ou não abstratos
 - Ex: A classe abstrata “Carro” define
- .os atributos da classe Carro
- .os métodos comuns da classe Carro
- .os métodos abstratos da classe Carro

*Classes Abstratas

- .Se não pode ser instanciada, serve pra que?

 - Polimorfismo

- .Quando uma subclasse redefine um método herdado

 - Herança de atributos e métodos

Exercício

- Crie uma classe “Animal” com atributo “Nome” e método “fala”.
- Faça as classes Homem, Cachorro e Gato herdando de “Animal”. Redefina o método “fala” para retornar “oi”, “au” e “miau”, respectivamente.
- Crie uma lista com 10 animais, cada um uma instância de Homem, Cachorro ou Gato (sortido, você escolhe).
- Faça um laço para exibir a seguinte frase “<nome do animal> fala <chama método fala>”

Especificação x Projeto

- A orientação a objetos em geral diminui a distância entre as fases de projeto e desenvolvimento
- Ela também facilita o processo de projeto a partir dos requisitos

Especificação x Projeto

- Não há um consenso acerca de onde termina a especificação e começa o projeto.
- A diferença está na ênfase
 - Especificação: entender o problema
 - Projeto: decidir como implementar os requisitos
- Ambos são representados através de modelos e precisam dos relacionamentos entre as funcionalidades para entender o sistema

Modelos

- São uma abstração da realidade
- Focam no que interessa para um determinado observador em um dado momento
- Por que usar modelos?
 - Possibilita a comunicação entre pessoas
 - Lidar com problemas complexos
 - Testar hipóteses antes de realizar

Modelos

.Ejemplos de modelo

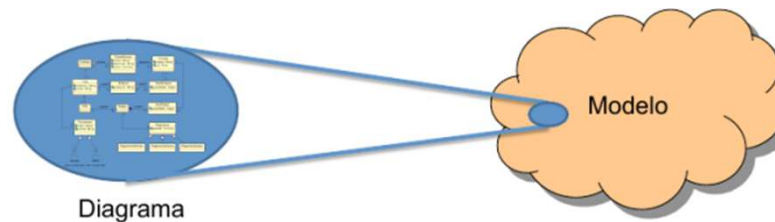
- Maquetes
- Manequins
- Plantas
- Esquemas
- Fórmulas

Modelos

- Todos os domínios do conhecimento usam modelos
- Idealmente, o modelo deve nos ajudar na derivação de outros modelos ou na codificação do sistema
 - Se não serve pra nenhum dos dois talvez não seja necessário

Modelos x Diagramas

- O modelo contém toda informação que representa o problema (ou a solução)
- O diagrama é uma visualização de parte do modelo sob uma certa perspectiva
 - Se está no diagrama, está no modelo
 - Se não está no diagrama, pode ou não estar no modelo



Exercício

- Modele um sistema de agenda usando os recursos que você achar relevante.
- (Talvez seja interessante levantar os requisitos com a turma toda)
 - Quais as vantagens da notação que você usou?
 - Quais as desvantagens?
 - Teve algo que você não conseguiu representar com a sua notação?

Modelagem de Software

- Criação de modelos que representam uma visão ou perspectiva diferente do sistema
- Normalmente usamos alguma notação gráfica, atualmente a notação utilizada é quase sempre a Unified Modeling Language (UML).
- Ajuda a entender a funcionalidade e se comunicar com os clientes

Tipos de Modelos

- **Modelo de fluxo de dados**

- Mostra como os dados são processados

- **Modelo de composição**

- Mostra como entidades são formadas a partir de outras

- **Modelo de arquitetura**

- Mostra os principais sub-sistemas

- **Modelo de classificação**

- Mostra características comuns entre entidades

Modelos de Contexto

- São aplicados no estágio inicial da análise de requisitos
- Ilustram o contexto operacional
 - Mostram o que está fora dos limites do sistema
- Implica em trabalhar junto com os stakeholders para definir o que é sistema e o que é ambiente

Exemplo: BIS sistema para Biblioteca

- Objetivo: fornecer versões eletrônicas (com direitos autorais) de livros para os usuários

- Assim os usuários podem imprimir cópias pessoais

- Pergunta: os outros sistemas da biblioteca estão dentro do limite?

- Ex: o sistema de catálogo

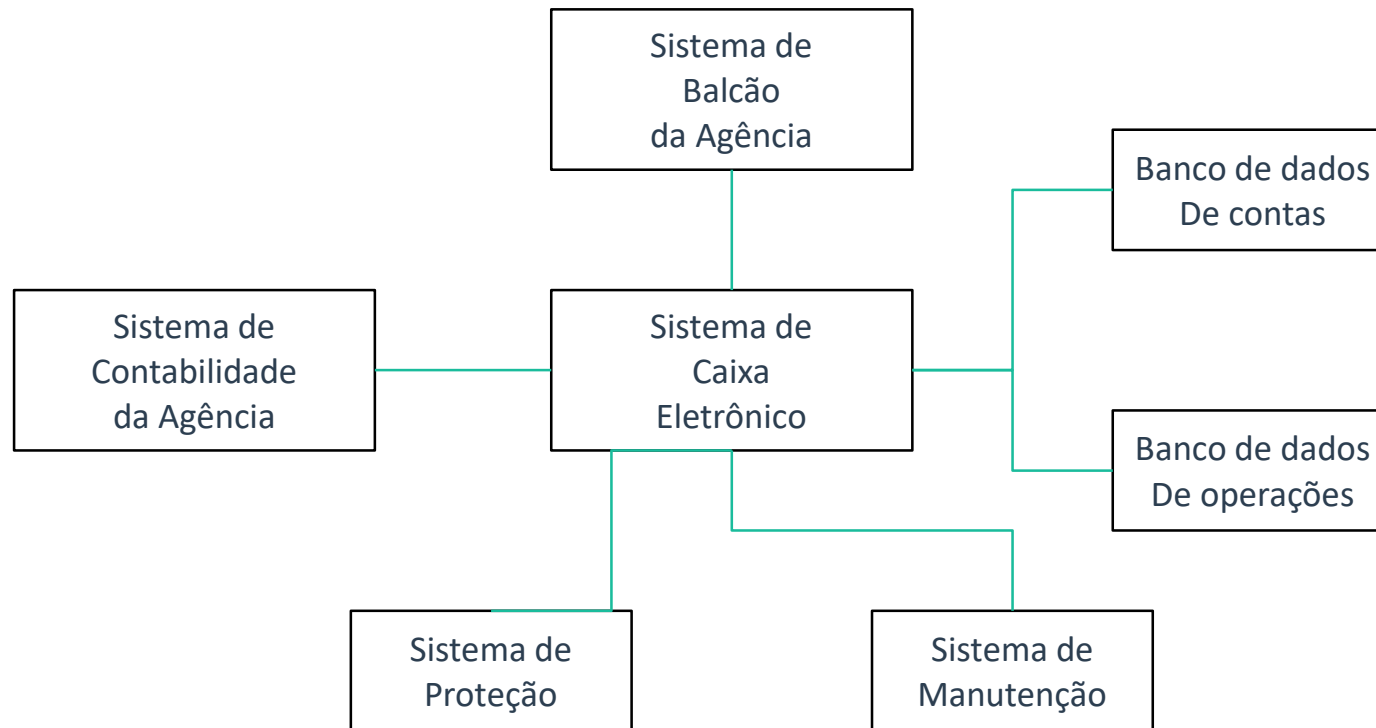
- Se sim, é preciso permitir acesso ao sistema por meio de uma interface com o usuário de catálogo

- Se não, o usuário sofre o incômodo de ter que mudar de um sistema para o outro

Modelos de Contexto

- Preocupações sociais e organizacionais podem afetar a decisão sobre onde posicionar as fronteiras do sistema
 - Pode evitar que um gerente chato não precise ser consultado
- Após as decisões do limite, produzimos um modelo simples de arquitetura
 - Mostra o sistema e sua relação com outros sistemas

Exemplo: Contexto de um Caixa Eletrônico



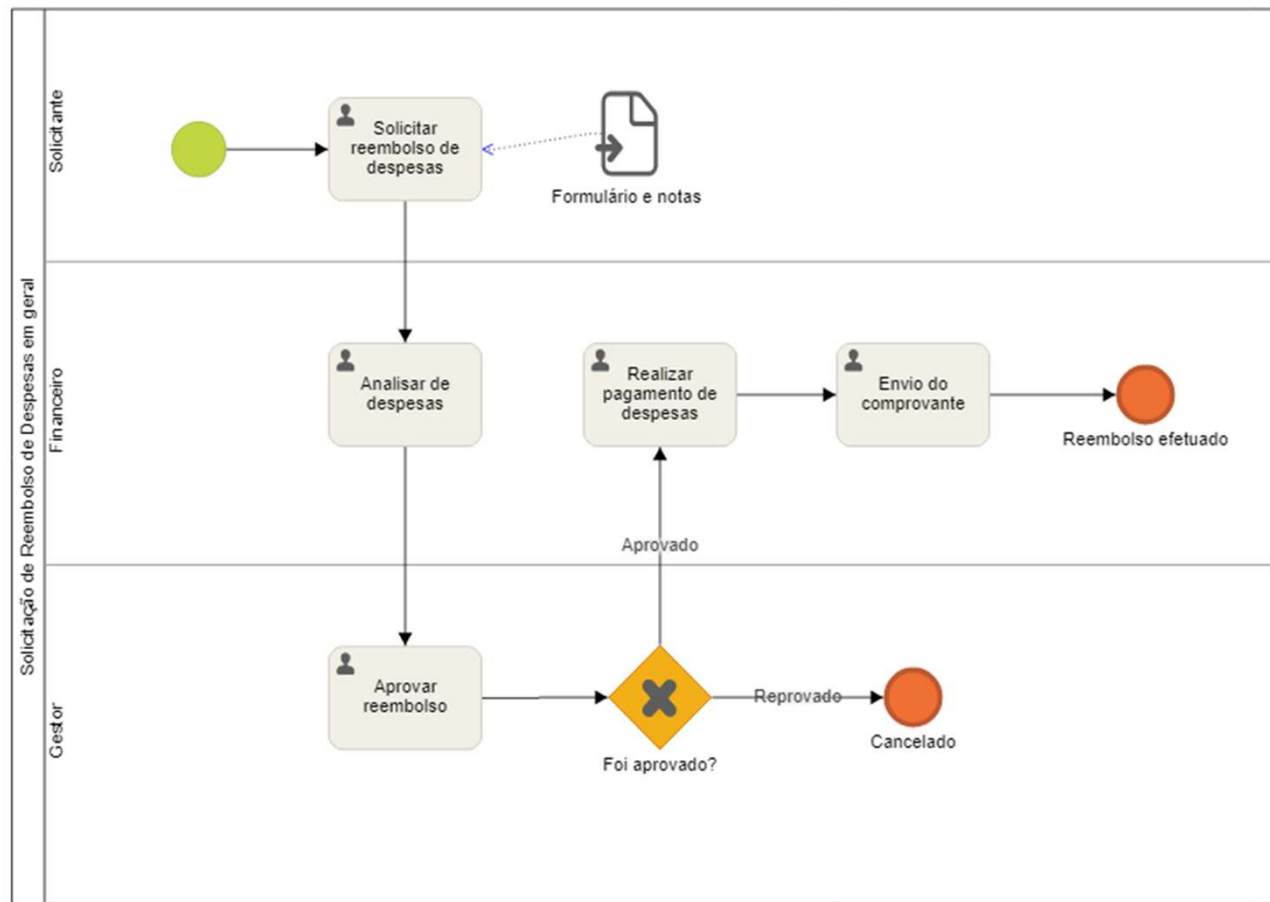
Modelos de Processo

- .Modelos de arquitetura não mostram a relação entre os outros sistemas no ambiente
- .Sistemas externos podem produzir dados ou consumir dados do sistema
- .Modelos de arquitetura são complementador por modelos de processo

Modelos de Processo

- Modelos de processo mostram o processo geral do sistema e os processos suportados por ele
- Modelos de fluxo de dados também podem ser utilizados para mostrar o fluxo de informações de um processo para o outro

Modelo de Processo - Exemplo



Modelos Comportamentais

- Descrevem o comportamento geral do sistema

- Tipos principais

- Fluxo de Dados: Como os dados são processados e como se move através do sistema

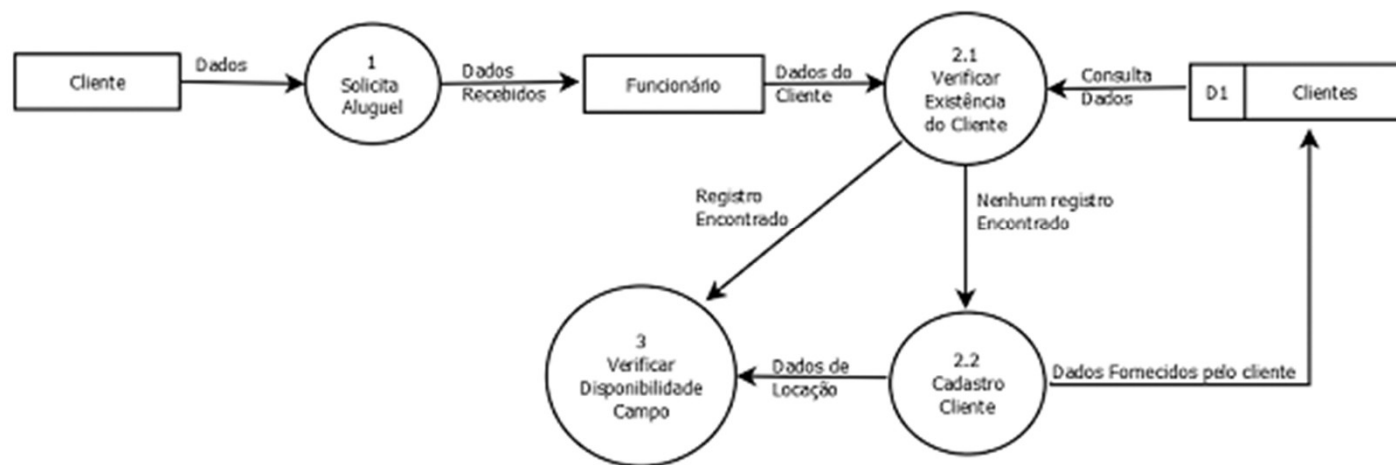
- Máquina de Estados: Mostra o sistema em resposta a eventos

- Os tipos principais mostram perspectivas distintas, ambos são necessários para uma descrição completa do comportamento do sistema

Modelo de Fluxo de Dados

- Podem ser usados para mostrar o processamento ponta a ponta em um sistema
- Mostram a sequência completa das ações que ocorrem
 - A partir de uma entrada
 - Que é processada
 - Até produzir a saída

Fluxo de Dados - Exemplo



Notação

.Geralmente Usamos

- Setas para o fluxo de dados
- Ovais ou retângulos arredondados para processos
- Retângulos para entidades externas
- Retângulos rotulados com Dx para repositórios de dados

Modelo de Máquina de Estados

- Modelam a resposta do sistema a eventos internos e externos
- Mostram os estados do sistema como nós (geralmente retângulos arredondados) e eventos com arcos (setas) entre os nós. Quando um evento ocorre o sistema muda de um estado para o outro.

Modelo de Máquina de Estados

- A cada instante o sistema deve estar em um, e somente um, dos estados.
- Ao receber um estímulo, pode disparar uma transição para um novo estado.
- Ao entrar num estado é possível executar uma ação.
- Ex: Uma lâmpada pode passar do estado “acesa” para o estado “apagada” em resposta ao evento “interruptor acionado”.

Diagrama de Estados - Exemplo

•Um micro-ondas tem os botões:

- Definir potência (total e média)
- Definir tempo
- Iniciar cozimento
- Cancelar operação

•Para usar o micro-ondas

- Selecione a potência (meia ou total)
- Insira o tempo de cozimento

Diagrama de Estados - Exemplo

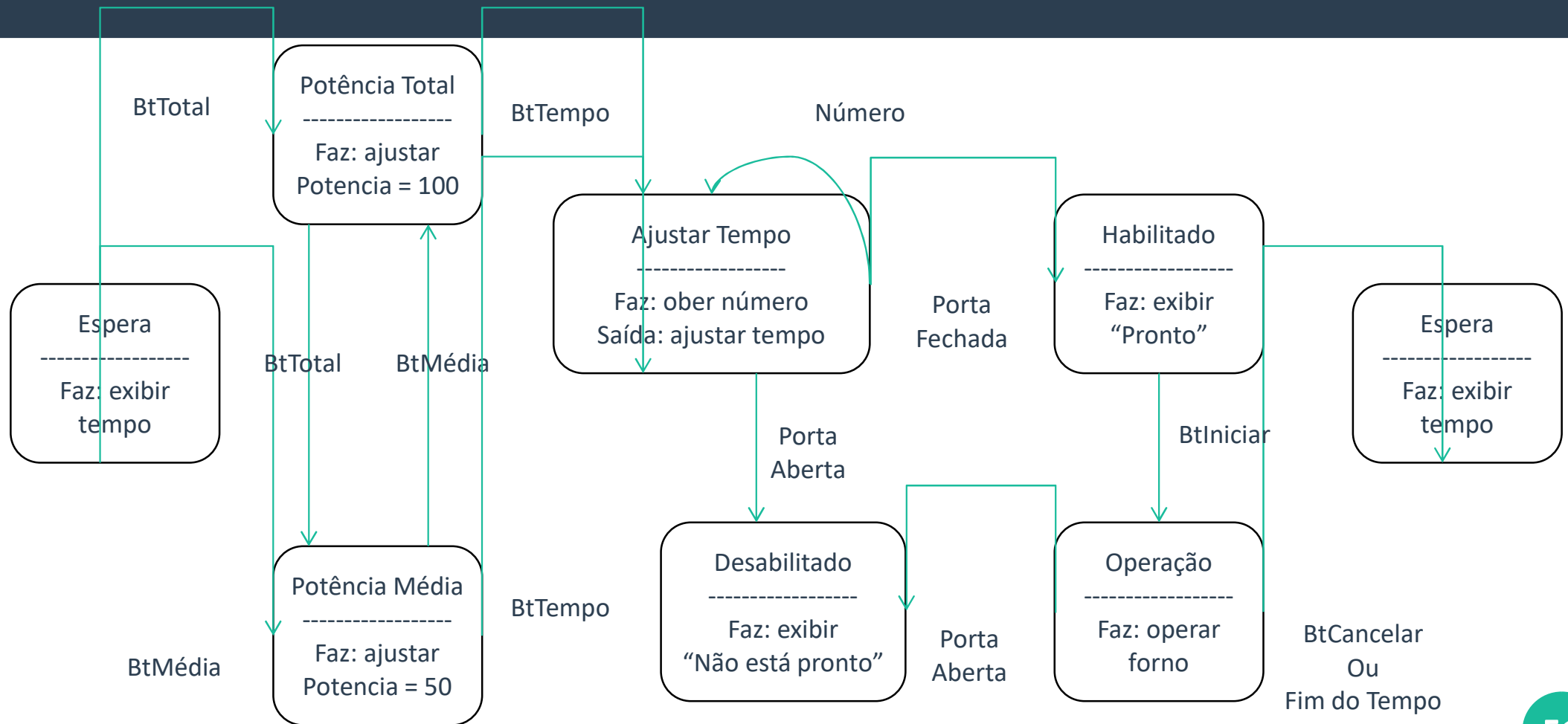


Diagrama de Estados - Exemplo

Descrição dos Estados	
Espera	O forno está aguardando a programação. Visor exibe a hora.
Potência Média	Ajusta a potência para 50%. Visor exibe potência média.
Potência Total	Ajusta a potência para 100%. Visor exibe potência total.
Ajustar Tempo	O tempo de cozimento é ajustado pelo usuário. Visor mostra o tempo de cozimento selecionado e é atualizado a cada número inserido.
Desabilitado	A operação do forno está desabilitada por questões de segurança. Luz interna acesa. Visor exibe “Não está pronto”
Habilitado	A operação do forno está habilitada. Luz interna apagada. Visor exibe “Pronto”
Operação	Forno em operação. Luz interna acesa. Exibe a contagem do tempo. Ao termino, alarme soa. Luz apaga. Visor exibe “concluído”

Diagrama de Estados - Exemplo

Descrição dos Eventos	
BtTotal	Usuário pressionou o botão potência total
BtMédia	Usuário pressionou o botão potência média
BtTempo	Usuário pressionou o botão de definir o tempo
Número	Usuário pressionou uma tecla numérica
Porta Aberta	A porta do micro-ondas está aberta
Porta Fechada	A porta do micro-ondas está fechada
BtIniciar	Usuário pressionou o botão iniciar cozimento
BtCancelar	Usuário pressionou o botão cancelar operação
Fim do Tempo	Contagem regressiva do tempo de cozimento chegou ao fim

Modelo de Estados

- .O número de estados pode ser muito grande**
- .Crie superestados, que englobam vários estados**
- .Cada superestado tem um diagrama de estados**

Modelos de Dados

- Descreve a estrutura lógica dos dados
- ERA (entidade – relacionamento – atributo) é a técnica mais usada
 - Define as entidades do sistema, os relacionamentos entre elas e os atributos das entidades
- Muito usada no projeto de banco de dados
- Também chamado de modelo semântico dos dados

Dicionário de Dados

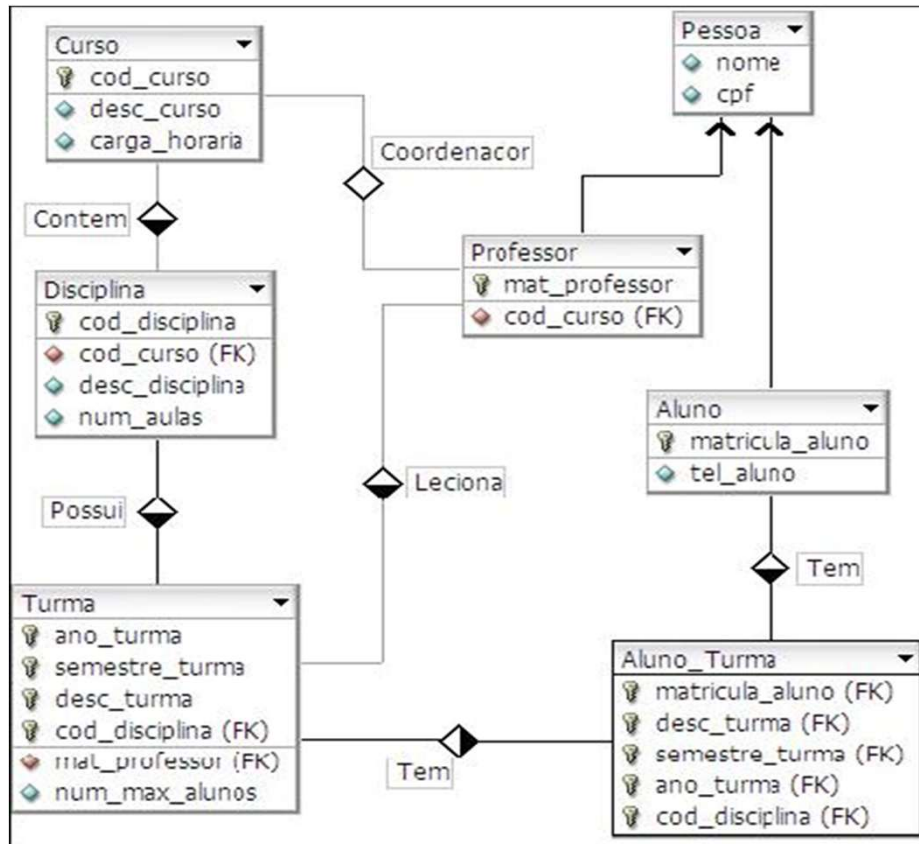
.São listas de todos os nomes usados nos modelos de sistema. Descrições das entidades, dos relacionamentos e dos atributos também são incluídos.

.Vantagens

- Evita duplicação de nomes

- Serve com repositório de informações do desenvolvimento, ligando análise, projeto, implementação e evolução

Modelo de Dados - Exemplo



Mais sobre isso quando virmos
Banco de dados

Modelo de Objetos

- Descrevem o sistema em termos de classes de objetos e suas associações
- As vezes identificar quem são as classes é um processo difícil
- Vários modelos de objetos:
 - Hierarquia
 - Agregação
 - Iteração

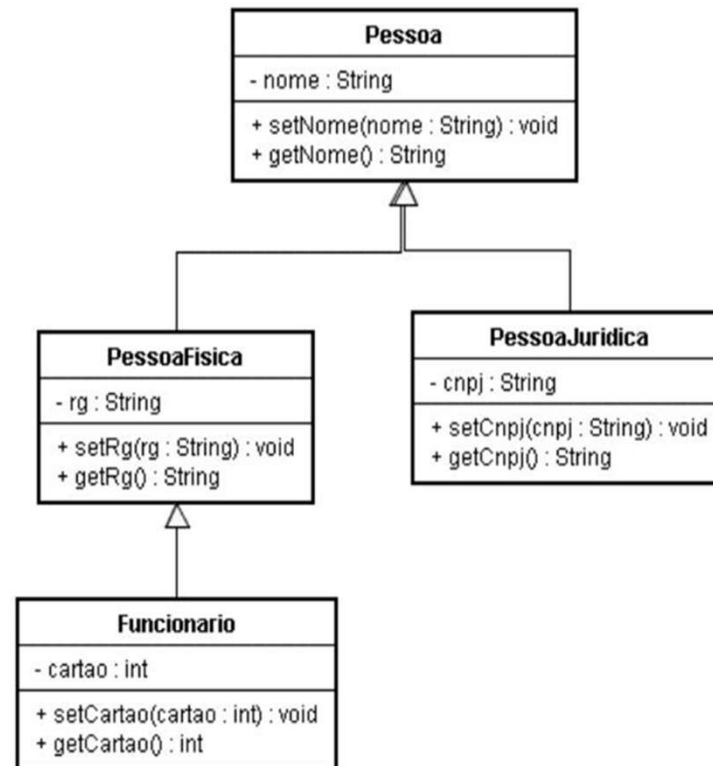
Modelo de Objetos

- A UML é a representação mais usada em projeto orientado a objetos
- Notação
 - Classes são retângulos com nome na parte superior, atributos na seção intermediária e métodos na seção inferior
 - Relacionamentos (ou associações) são linhas que ligam os objetos
 - Herança é chamada de generalização e mostrada no sentido ascendente

Modelo de Objetos - Hierarquia

- Organiza classes em uma hierarquia
- A classe no topo refletem as características comuns de todas as classes abaixo dela
- As classes de objeto herdam seus atributos e métodos de uma ou mais super classes. Podem então ser especializadas, se necessário

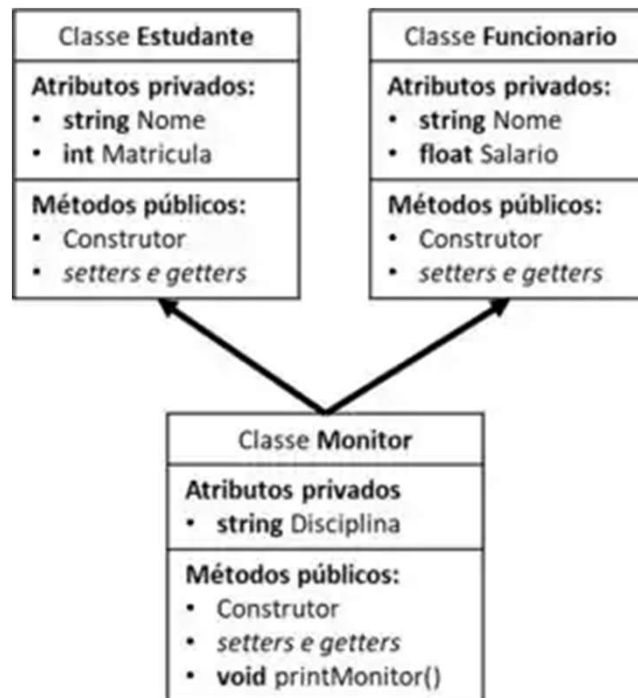
Hierarquia - Exemplo



Herança Múltipla

- Um sistema que suporta herança múltipla permite que classes herdem de várias super-classes
- Isso pode levar a conflitos semânticos onde atributos ou métodos com o mesmo nome em diferentes super-classes têm significados distintos
- A herança múltipla torna a reorganização da hierarquia mais complexa

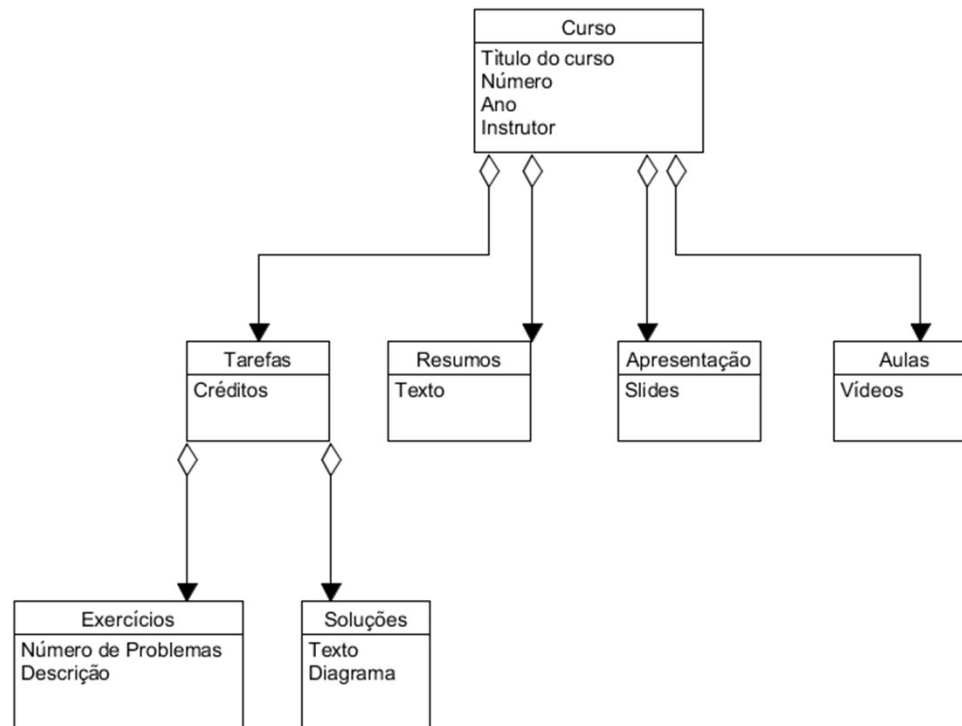
Herança Múltipla - Exemplo



Modelo de Objetos - Agregação

- Mostra como as classes que são coleções são compostas de outras classes
- Modelos de agregação podem ser lidos como “é parte de”.
- Também ocorre em modelos semânticos ou fluxo de dados

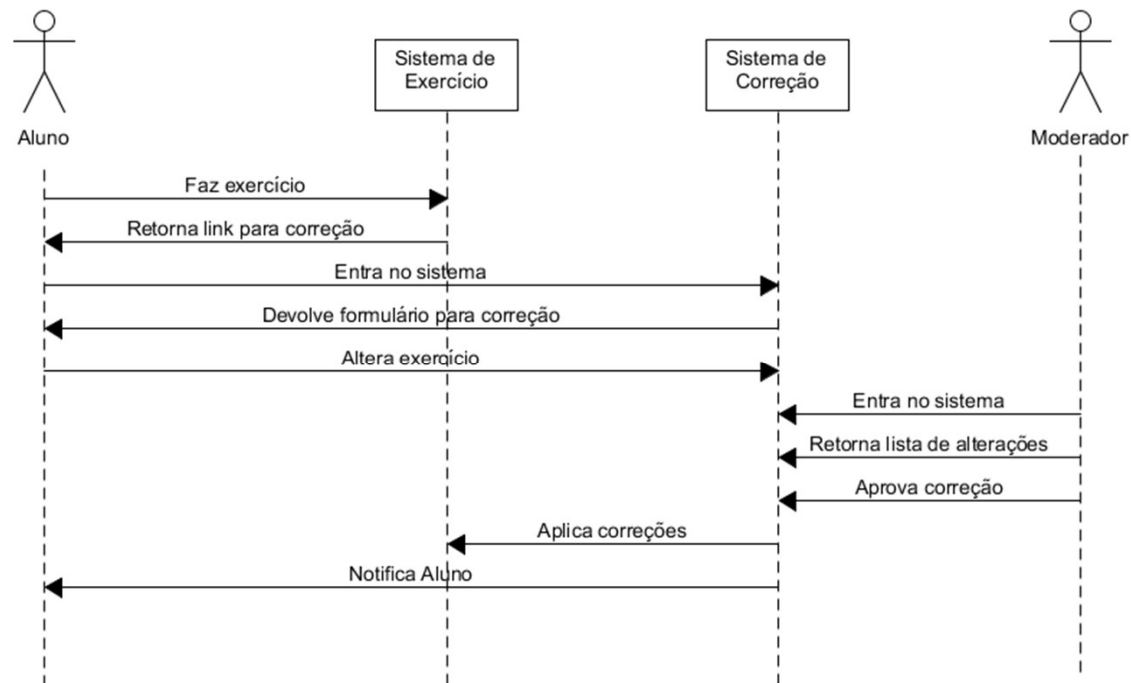
Agregação - Exemplo



Modelagem de Comportamento de Objetos

- Mostra as interações entre objetos que produzem algum comportamento do sistema em particular que é especificado como caso de uso
- Diagramas de sequência (ou diagramas de colaboração) são usados para modelar a interação entre objetos

Diagrama de Sequência - Exemplo



Considerações Finais

- .São muitos modelos, se fizer tudo é provável que tenha um excesso de documentação, use o que for necessário**
- .Os modelos de sistema às vezes são muito detalhados e difíceis para os usuários compreenderem**